Unit IV

Graphics Primitives, Scan Conversion,Windowing and Clipping

Computer Graphics

Computer graphics is a field of computer science that focuses on creating, manipulating, and displaying visual content using computers. This includes everything from simple 2D images to complex 3D models and animations.

It displays the information in the form of graphics objects such as pictures, charts, graphs and diagrams instead of simple text. Hence, we can say that computer graphics makes it possible to express data in pictorial form.

Applications of Computer Graphics

- **Video Game Development**: Used to create visually rich and immersive game environments,character models, and dynamic effects in both 2D and 3D video games.

- **Movie and Animation Production:** Essential for generating special effects, animated characters, and realistic environments in films, TV shows, and animated features, including CGI (Computer–Generated Imagery).

- **Computer–Aided Design (CAD):** Widely used in engineering, architecture, and manufacturing to design and visualize products, buildings, and mechanical systems before physical production.
- **Medical Imaging:** Important for visualizing complex medical data from MRI, CT scans, and other imaging technologies, aiding in diagnosis, surgical planning, and medical research.

https://t.me/SPPU_TE_BE_COMP

Study material provided by: Vishwajeet Londhe

Join Community by clicking below links

Telegram Channel

https://t.me/SPPU_TE_BE_COMP

(for all engineering Resources)



WhatsApp Channel

(for all Engg & tech updates)

https://whatsapp.com/channel/0029ValjFriICVfpcV9HFc3b



Insta Page

(for all Engg & tech updates)

https://www.instagram.com/sppu_engineering_update

- **Presentation Graphics:** For the preparation of reports or summarizing the financial, statistical, mathematical, scientific, and economic data for research reports, and managerial reports, moreover creation of bar graphs, pie charts, and time charts, can be done using the tools present in computer graphics.

- **Machine Drawing:** Computer graphics are very frequently used for designing, modifying, and creating various parts of a machine and the whole machine itself.

- **Graphical User Interface**: The use of pictures, images, icons, pop–up menus, and graphical objects helps in creating a user–friendly environment where working is easy and pleasant, using computer graphics we can create such an atmosphere where everything can be automated and anyone can get the desired action performed in an easy fashion.

Graphics Primitives

Pixel

- A pixel is the smallest unit of information in an image or display on a computer, smartphone, or television.
- Pixels are tiny squares or dots that represent a single point of color. They are usually arranged in a regular 2–dimensional grid.
- Each pixel is made up of a red, green, and blue (RGB) subpixel that emits color at different intensities.

Resolution

Resolution refers to the amount of detail or clarity of the display, measured in pixels. It indicates the number of pixels along the width

https://t.me/SPPU_TE_BE_COMP

and height of a screen or image. Higher resolutions have more pixels and, therefore, can display more detail and sharper images.

Some common resolutions are:

640x480 – VGA (Video Graphics Array)

800x600 – SVGA (Super VGA)

1280x720 (720p) – HD (High Definition)

1920x1080 (1080p) – Full HD (FHD)

Aspect Ratio

Aspect ratio is the ratio of the width to the height of an image, screen, or display. It is usually expressed as two numbers separated by a colon (e.g., 16:9). This ratio defines how the width compares to the height, and it helps determine the shape of the image or screen.

Some common aspect ratios are:

4:3 – Standard Definition (SD)

16:9 – High Definition (HD) / Full HD (FHD) / Ultra HD (UHD)

Frame Buffer

A frame buffer is a portion of memory used to store image data that will be displayed on a screen. It acts as a temporary storage area where the graphical data is kept before it is sent to the display.

The frame buffer holds the color and pixel information for each point on the screen,allowing for smooth and consistent image rendering.

https://t.me/SPPU_TE_BE_COMP

Display devices

A display device is the output device that shows pictures, text, or videos on a screen.

It is what we see when we use a computer, TV, mobile, or projector.

In computer graphics, display devices help us to view the results of drawing, animations, or any graphics work.

Types of Display Devices

1. Cathode Ray Tube (CRT)

2. Direct View Storage Tube (DVST)

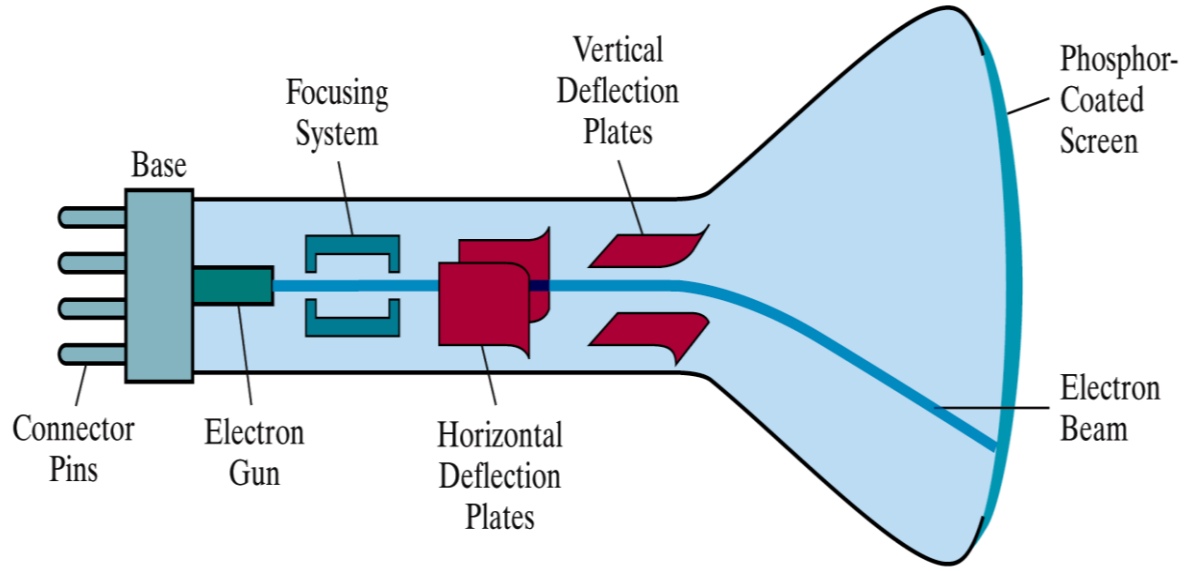
3. Flat Panel Display

https://t.me/SPPU_TE_BE_COMP

Cathode-ray Tube (CRT):

Here, CRT stands for Cathode ray tube. It is a technology which is used in traditional computer monitor and television.

Cathode ray tube is a particular type of vacuum tube that displays images when an electron beam collides on the radiant surface.



Component of CRT

1. **Connector Pins** – Connect CRT to the external circuit.
2. **Electron Gun** – Produces a narrow beam of electrons.
3. **Focusing System** – Focuses the electron beam into a sharp point.
4. **Horizontal & Vertical Deflection Plates** – Control the direction of the electron beam (move it left-right and up-down).
5. **Electron Beam** – The stream of electrons that forms the picture.

https://t.me/SPPU_TE_BE_COMP

6. **Phosphor-Coated Screen** – Glows when hit by electrons → displays the image.
7. **Glass Enclosure (Tube/Base)** – Vacuum tube that holds all components.

Working of CRT (Cathode Ray Tube)

1. **Electron Emission (Electron Gun)**

The **electron gun** emits a stream of electrons by heating a cathode. These electrons are accelerated using anodes to form a high-speed **electron beam**.

2. **Beam Focusing (Focusing System)**

The electron beam passes through a **focusing system**, which narrows it down into a fine, sharp beam for clear image formation on the screen.

3. **Beam Deflection (Deflection Plates)**

The beam then passes through two sets of **deflection plates**:

- **Vertical deflection plates** control **horizontal movement** (left-right).
- **Horizontal deflection plates** control **vertical movement** (up-down).

4.Beam Bending by Voltage

The beam is **bent** or **deflected** by **increasing or decreasing the voltage** applied to the deflection plates.

- **Higher voltage** → more bending
- **Lower voltage** → less bending
  By carefully changing these voltages, the beam can be directed to any point on the screen.

https://t.me/SPPU_TE_BE_COMP

5. Beam Strikes the Screen (Phosphor-Coated Screen)

The deflected beam hits the **phosphor-coated screen** at the front of the CRT. The point where the beam hits starts to **glow** (emit light).

6.Image/Waveform Formation

By continuously changing the deflection voltages and controlling the beam's intensity, the CRT can draw **lines, shapes, waveforms, or full images** on the screen.

Advantages of CRT

1. **Good Image Quality** – CRTs provide clear and sharp images.
2. **Fast Response Time** – No lag; suitable for fast-moving images.
3. **Multiple Resolutions Supported** – Can work at various screen resolutions.
4. **Wide Viewing Angle** – Picture looks good even from the sides.
5. **Low Cost** – Cheaper compared to modern display technologies.

Disadvantages of CRT

1. **Bulky and Heavy** – CRTs are large and hard to carry.
2. **High Power Consumption** – They use more electricity.
3. **Heat Generation** – CRTs produce a lot of heat during use.
4. **Harmful Radiation** – Emits low-level radiation (old models).
5. **Fragile Screen** – Glass screen can break easily

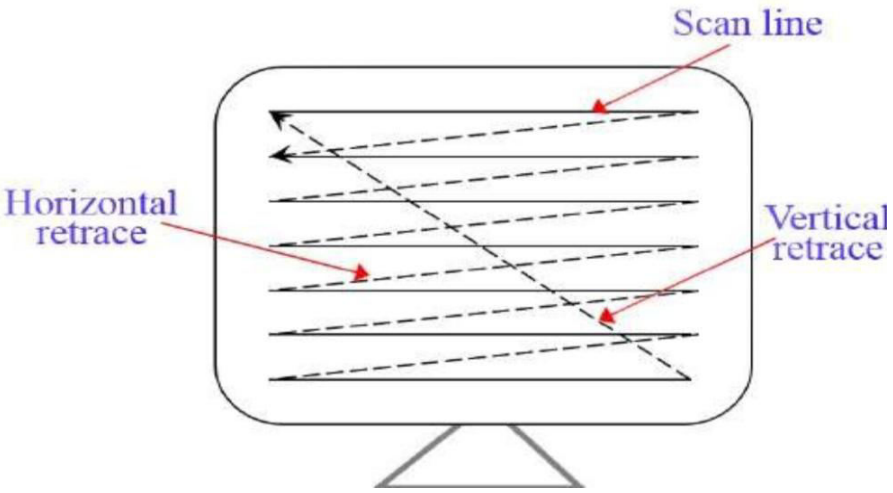
There are two techniques used for producing images on the CRT screen—

1. Raster Scan display
2. Vector scan / Random Scan display

https://t.me/SPPU_TE_BE_COMP

1.Raster Scan display

Raster Scan is a technique where the electron beam scans the screen **line by line from top to bottom**, and each line is made up of **pixels**.



Working of the Raster Scan display

1. Each **scan line** can be thought of as a **horizontal line of pixels** on the screen.
2. The scanning process starts from the **top-left corner** of the screen.
3. Each pixel is **illuminated** according to its **intensity value**, which is stored in the **refresh/frame buffer**.
4. The **picture definition** (image) is stored in the form of intensity values in the buffer, from where it is **retrieved and displayed** on the screen.
5. When the **electron beam sweeps from left to right** along a scan line:
    a. The beam is in **ON mode**.
    b. It **illuminates the pixels** according to their intensity values.

https://t.me/SPPU_TE_BE_COMP

6. When the transition is made to the **next scan line**:
    a. The beam is turned **OFF**.
    b. The beam moves **from right to left** — this movement is called **horizontal retrace**.
7. After reaching the **bottom-right corner** of the screen:
    a. The beam moves diagonally to the **top-left corner**.
    b. This movement is called **vertical retrace**.
    c. During this retrace, the beam remains **OFF**.

Advantages of Raster Scan

1. **Can display complex images** like photos, videos, and filled shapes.
2. **Works with pixels**, making it suitable for color and shading.
3. **Cheaper and simpler hardware** compared to random scan.
4. **Widely used** in TVs, monitors, and computer screens.

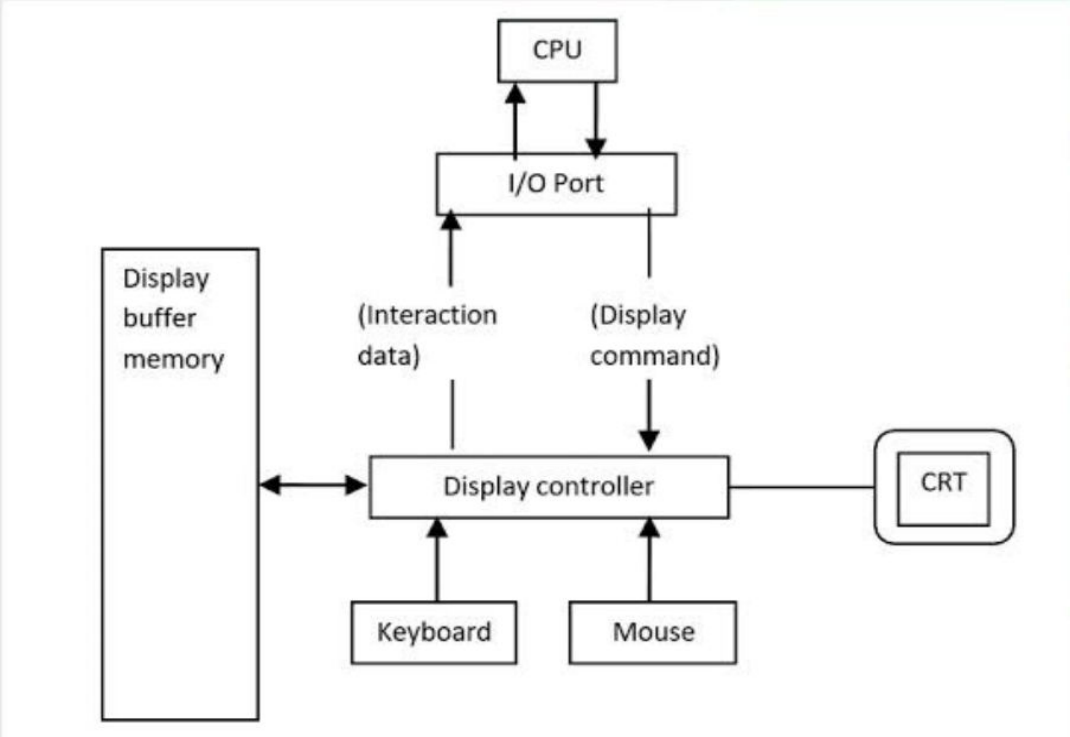
Disadvantages of Raster Scan

1. **Lines and curves are less sharp** (due to pixelation).
2. **Uses more memory** (needs a frame buffer to store pixel data).
3. **Slower for line drawings** compared to Random Scan.

https://t.me/SPPU_TE_BE_COMP

2. Random Scan Display

Random Scan Display is a display system where the electron beam is directed to draw images by tracing only t**he necessary lines, rather than scanning the whole screen.** It is used mainly for line-based drawings and is also called **Vector Scan Display**.



Working of Random Scan Display (Vector Scan Display)

1. The **CPU** sends drawing instructions (for lines, points, or characters) to the **Display Controller**.
2. The **Display Controller** receives and stores these instructions in a special memory called the **Display Buffer** (or **Display File**).
3. The **Display Controller** reads commands from the display buffer and directs the **electron beam** to move across the screen **only along the required lines**, not in a fixed raster pattern.

https://t.me/SPPU_TE_BE_COMP

4. The beam moves between specified points, drawing **lines directly** by activating the beam **only when needed**.
5. The screen is **phosphor-coated**, so when the beam hits a spot, it **glows temporarily**.
6. Over time, this glow **fades gradually** due to the **decay of the phosphor effect**.
7. To keep the image visible, the **same drawing commands are repeated continuously** by the display controller.
8. This repeated drawing is called **refreshing**, and it typically happens at a rate of **30 to 60 times per second (30–60 Hz)**.
9. The display shows only the lines and shapes as directed, making it suitable for **wireframes, engineering drawings, graphs**, etc.

Advantages of Random Scan Display

1. **Smooth lines** – Lines and curves look smooth (no pixelation).
2. **Memory efficient** – Only stores drawing commands, not every pixel.
3. **High resolution for line drawings** – Great for technical graphics and CAD.

Disadvantages of Random Scan Display

1. **Only supports line drawings** – Cannot display complex images like photos.
2. **Slow for complex scenes** – More lines = more time to refresh.
3. **Refreshing required** – Image fades, so needs continuous redrawing.
4. **Outdated technology** – Replaced by raster scan in modern displays.

https://t.me/SPPU_TE_BE_COMP

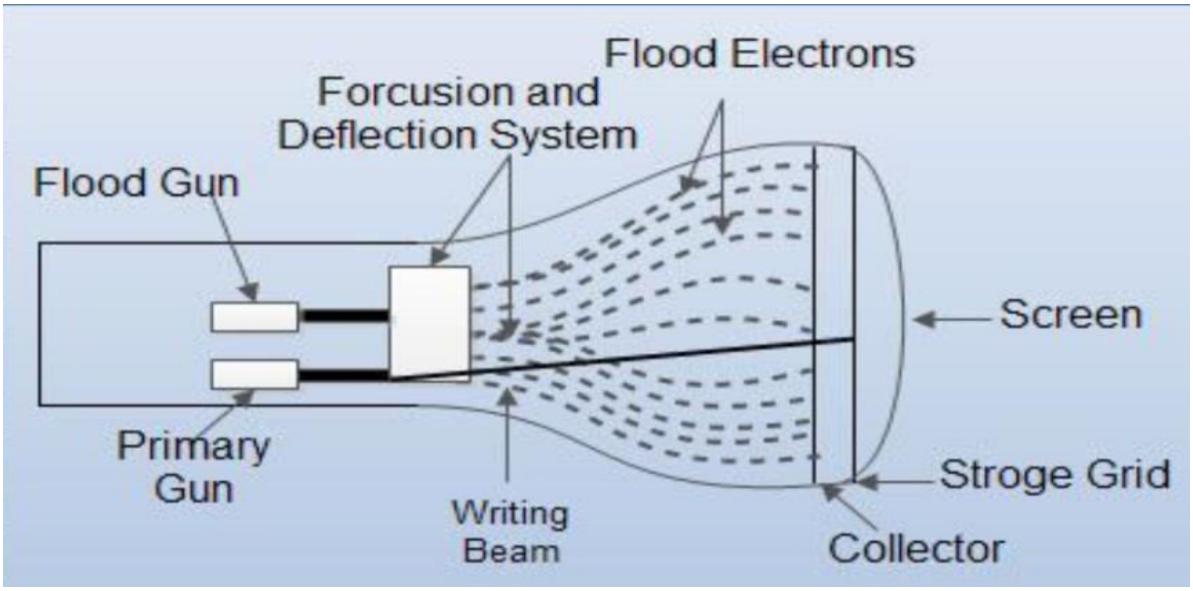
Comparison between Random Scan and Raster Scan

Sr No	Random Scan	Raster Scan
1	Represents images as a collection of lines, curves, and other geometric shapes (vector graphics).	Represents images as a collection of pixels arranged in rows and columns (raster graphics).
2	Uses a beam of electrons to draw lines directly on the screen in any order.	The electron beam sweeps across the screen from left to right and top to bottom to illuminate the pixels.
3	Resolution–independent; the quality of images doesn't degrade when scaled.	Resolution–dependent; the quality of images can degrade when scaled due to pixelation.
4	Variable refresh rate, depending on the complexity of the image.	Fixed refresh rate, typically 60Hz or higher, to maintain consistent image display.
5	Ideal for applications involving line drawings, CAD, and simple graphics.	Suitable for rendering complex images, including photos, videos, and detailed graphics.
6	Requires less memory since only the coordinates of the endpoints of lines are stored.	Requires more memory to store the entire frame buffer, which holds the pixel data for the entire screen.

https://t.me/SPPU_TE_BE_COMP

2. Direct View Storage Tube (DVST)

DVST is a type of display device that stores the image directly on the screen without the need for refreshing.



Working of DVST (Direct View Storage Tube)

1. **DVST is a type of display device** that stores the image directly on the screen without the need for continuous refreshing like raster scan systems.
2. The screen of DVST is coated with **phosphor** that **glows when struck by an electron beam**.
3. DVST contains two electron guns:
    - **Primary gun**: Used for drawing the image.
    - **Flood gun**: Provides a steady stream of electrons to maintain the display.
4. The image is drawn by the **primary gun**, which emits a focused beam to **write directly on the storage mesh** behind the phosphor-coated screen.
5. The **storage mesh** holds the **positive charge pattern** of the image where the primary beam hits.

https://t.me/SPPU_TE_BE_COMP

6. The **flood gun** emits electrons that are attracted to the positively charged areas of the storage mesh.
    - These electrons pass through the mesh and strike the phosphor screen, causing it to glow.
7. The phosphor has a **long persistence**, so the image **remains visible for several minutes** without refreshing.
8. To **erase the image**, a process called **"erasure"** is performed by applying a uniform negative charge to the mesh, removing all stored charges.
9. New images can then be drawn by the primary gun after the screen is cleared.

Advantages of DVST (Direct View Storage Tube)

1. **No need for refreshing** – The image stays on screen for a long time without flickering.
2. **Clear and stable display** – The picture doesn't flicker or shake.
3. **Good for static images** – Ideal for diagrams, graphs, and text.

Disadvantages of DVST

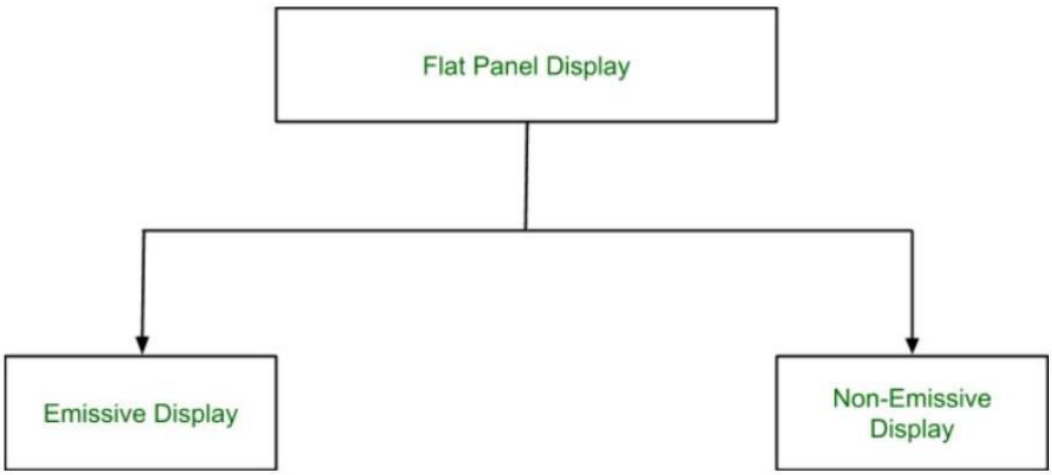
1. **No animation possible** – You can't update part of the image.
2. **Slow update** – To change anything, you must clear and redraw the entire screen.
3. **Not suitable for dynamic graphics** – Can't be used for games or videos.

https://t.me/SPPU_TE_BE_COMP

3. Flat Panel Display

Flat-Panel Devices are the devices that have less volume, weight, and power consumption compared to Cathode Ray Tube (CRT). Due to the advantages of the Flat-Panel Display, use of CRT decreased. As Flat Panel Devices are light in weights that's why they can be hang on walls and wear them on our wrist as a watch. Flat Panel Display (FPD) allow users to view data, graphics, text and images.

Types of Flat Panel Display:



1. Emissive Display:

The Emissive Display or Emitters are the devices that convert electrical energy into light energy.

Examples: Plasma Panel, LED (Light Emitting Diode), Flat CRT.

https://t.me/SPPU_TE_BE_COMP

2. Non-Emissive Display:

Non-Emissive Display or Non-Emitters are the devices that use optical effects to convert sunlight or some other source into graphic patterns.

Examples: LCD (Liquid Crystal Display)

Advantages of Flat Panel Devices:

- Flat Panel Devices like LCD produces high quality digital images.
- Flat Panel monitor are stylish and have very space saving design.
- Flat Panel Devices consumes less power and give maximum image size in minimum space.
- Flat Panel Devices use its full color display capability.
- Full motion video can be viewed on Flat Panel Devices without artifacts or contrast loss.

Disadvantages of Flat Panel Devices:

- They are very expensive compared to CRTs.
- They have very high refresh rates.
- Slow response times.
- They may be heavier and bulkier than other display types.

Scan conversion

Scan conversion is the process of **converting geometric shapes (points, lines, curves, circles) from their mathematical form into pixels on a raster display** so they can be shown on the screen.

- It decides **which pixels to turn on** to represent the shape accurately.
- Example: Drawing a line between two points on a computer screen by filling the appropriate pixels.

https://t.me/SPPU_TE_BE_COMP

➢ Line drawing algorithm

1. **Digital Differential Analyzer (DDA) Line drawing algorithm**
2. **Bresenham's Line drawing algorithm**

1. Digital Differential Analyzer (DDA) Line drawing algorithm

The Digital Differential Analyzer (DDA) Line Drawing Algorithm is a popular algorithm used in computer graphics to draw lines between two points on a grid or screen.

The DDA algorithm is simple and efficient for generating lines with arbitrary slopes.

The idea behind the DDA algorithm is to calculate values between the start and end points by incrementing either the x–coordinate or the y–coordinate based on the line's slope.

The algorithm steps through one coordinate at a time (either x or y) and calculates the corresponding y or x value using the line's slope.

https://t.me/SPPU_TE_BE_COMP

Steps in the DDA Algorithm

1. **Specify the Start and End Points:**
   - Let the line start at point **(x_0, y_0)** and end at point **(x_1, y_1)**.

2. **Calculate Δx and Δy:**
   - Compute the horizontal and vertical distances between the start and end points:
     - **$\Delta x = x_1 - x_0$**
     - **$\Delta y = y_1 - y_0$**

3. **Determine the Number of Steps:**
   - The number of steps required to draw the line is the **maximum of |Δx| and |Δy|**:
     - **steps = max(|Δx|, |Δy|)**
   - This ensures the line is drawn with appropriate resolution.

4. **Calculate the Increments:**
   - Compute the amount by which the x and y values will be incremented in each step:
     - **$x_i = \Delta x$ / steps**
     - **$y_i = \Delta y$ / steps**

5. **Start Drawing the Line:**
   - Initialize the starting point:
     - **$x = x_0$, $y = y_0$**
   - For each step:
     - Plot the pixel at the current coordinates **(x, y)**
     - Update the coordinates:
       - **$x = x + x_i$**
       - **$y = y + y_i$**

6. **Round the Coordinates Before Plotting:**
   - Since screen pixels are at integer positions, **round x and y to the nearest integer** before plotting each point.

https://t.me/SPPU_TE_BE_COMP

Advantages of DDA

1. **Simple and Easy to Understand**
   The algorithm is easy to implement.
2. **Works for Any Slope**
   Can draw lines with **positive, negative, steep, or shallow slopes**.

Disadvantages of DDA:

1. The algorithm involves floating–point arithmetic, which can be slower and less efficient on systems without floating–point hardware.

2. The rounding step can introduce errors in drawing the line.

Consider a line segment from A(−2, −1) to B(6, 3). Use the DDA Line Drawing Algorithm to rasterize this line.

Step 1: Starting and Ending Points

- Starting point: $A(x_0, y_0) = (-2, -1)$
- Ending point: $B(x_1, y_1) = (6, 3)$

Step 2: Calculate Δx and Δy

- $\Delta x = (x_1 - x_0) = [6 - (-2)] = 6 + 2 = \mathbf{8}$
- $\Delta y = (y_1 - y_0) = [3 - (-1)] = 3 + 1 = \mathbf{4}$

Step 3: Determine the Number of Steps

- steps = max(|Δx|, |Δy|) = max(8, 4) = **8**

Step 4: Calculate the Increments

- $x_i = \Delta x$ / steps = 8 / 8 = **1**
- $y_i = \Delta y$ / steps = 4 / 8 = **0.5**

https://t.me/SPPU_TE_BE_COMP

Step 5: Start Drawing the Line

- Initialize starting point: (x, y) = (−2, −1)
- For each step:
  - Plot the pixel at the current (x, y) coordinates
  - Update coordinates:
    - $x = x + x_i$
    - $y = y + y_i$

Step 6: Round the Coordinates and Plot Each Point

Step	x	y	Plotted Point
1	−1	−1	(−1, −1)
2	0	0	(0, 0)
3	1	0.5	(1, 1)
4	2	1	(2, 1)
5	3	1.5	(3, 2)
6	4	2	(4, 2)
7	5	2.5	(5, 3)
8	6	3	(6, 3)

Using the DDA Algorithm compute the pixels that would be turned on for line with end points (0,0) to (3,2)

Step 1: Δx and Δy

$$\Delta x = 3 - 0 = 3, \quad \Delta y = 2 - 0 = 2$$

Step 2: Steps

$$\text{steps} = \max(\Delta x, \Delta y) = \max(3, 2) = 3$$

Step 3: Increments

$$x_{\text{inc}} = \frac{\Delta x}{\text{steps}} = \frac{3}{3} = 1$$

$$y_{\text{inc}} = \frac{\Delta y}{\text{steps}} = \frac{2}{3} \approx 0.667$$

https://t.me/SPPU_TE_BE_COMP

Step 4: Initialize starting point

$$x = x_0 = 0, \quad y = y_0 = 0$$

- Plot first pixel: (0,0)

Step 1:

- Previous x = 0, previous y = 0
- Add increments:

$$x = 0 + 1 = 1$$

$$y = 0 + 0.667 \approx 0.667$$

- Round y → 1
- **Pixel = (1,1)**

Step 2:

- Previous x = 1, previous y = 0.667
- Add increments:

$$x = 1 + 1 = 2$$

$$y = 0.667 + 0.667 \approx 1.334$$

- Round y → 1
- **Pixel = (2,1)**

https://t.me/SPPU_TE_BE_COMP

Step 3:

- Previous x = 2, previous y = 1.334
- Add increments:

$$x = 2 + 1 = 3$$

$$y = 1.334 + 0.667 \approx 2.001$$

- Round y → 2
- **Pixel = (3,2)**

Step 5: Generate remaining points (add increments & round)

Steps	x (approx)	y (approx)	Pixel (round)
1	0 + 1 = 1	0 + 0.667 ≈ 0.667	(1,1)
2	1 + 1 = 2	0.667 + 0.667 ≈ 1.334	(2,1)
3	2 + 1 = 3	1.334 + 0.667 ≈ 2.001	(3,2)

Step 6: Final pixels plotted
(0,0),(1,1),(2,1),(3,2)

Bresenham's Line Drawing Algorithm

Bresenham's Line Drawing Algorithm is an efficient method used in computer graphics to draw lines on a pixel–based display.

Unlike the DDA (Digital Differential Analyzer) algorithm, Bresenham's algorithm uses only integer arithmetic, making it faster and more suitable for systems that lack floating–point hardware.

https://t.me/SPPU_TE_BE_COMP

The algorithm determines the closest pixel to the theoretical line at each step, using incremental error terms to decide whether to move horizontally, vertically, or diagonally.

Step 1: Calculate Δx and Δy

- Compute the differences between the end and start points:
  - $\Delta x = x_1 - x_0$
  - $\Delta y = y_1 - y_0$

Step 2: Initialize Decision Parameter

- Define the initial **decision parameter p_0** based on the slope of the line:
  - $p_0 = 2\Delta y - \Delta x$
- This parameter helps determine which pixel to plot next.

Step 3: Plot the Initial Point

- Begin by **plotting the first point**:
  - (x_0, y_0)

Step 4: Iterate Over x (or y for steep lines) and Plot Points

- For each **x** from x_0 **to** x_1, perform the following:

  Case 1: If $p < 0$

  - The next point is $(x_k + 1, y_k)$
  - Update the decision parameter:
    - $p_{k+1} = p_k + 2\Delta y$

  Case 2: If $p \geq 0$

  - The next point is $(x_k + 1, y_k + 1)$
  - Update the decision parameter:
    - $p_{k+1} = p_k + 2\Delta y - 2\Delta x$

Repeat

- Continue this process **until the endpoint (x_1, y_1) is reached**.

https://t.me/SPPU_TE_BE_COMP

Example: Consider a line from (1, 1) to (6, 4). Use Bresenham's algorithm to rasterize this line.

Step 1: Calculate Δx and Δy:

$\Delta x = x1 - x0 = 6 - 1 = 5$
$\Delta y = y1 - y0 = 4 - 1 = 3$

Step 2: Initialize Decision Parameter:

$pk = 2\Delta y - \Delta x = 2(3) - 5 = 1$

Step 3: Plot the initial points:

Start by plotting the first point (1, 1).

Step 4: Iterate over x or y for steep lines and plot points:

$pk+1 = pk + 2\Delta y - 2\Delta x = 1 + 2(3) - 2(5) = -3$ = updated pk
$pk+1 = pk + 2\Delta y = -3 + 2(3) = 3$ = updated pk
$pk+1 = pk + 2\Delta y - 2\Delta x = 3 + 2(3) - 2(5) = -1$ = updated pk
$pk+1 = pk + 2\Delta y = -1 + 2(3) = 5$ = updated pk

Steps	pk	x	y	Plotted Point
1	1	2	2	(2, 2)
2	-3	3	2	(3, 2)
3	3	4	3	(4, 3)
4	-1	5	3	(5, 3)
5	5	6	4	(6, 4)

Final set of plotted points using Bresenham's algorithm:

(1, 1), (2, 2), (3, 2), (4, 3), (5, 3), (6, 4)

https://t.me/SPPU_TE_BE_COMP

Q.Consider a line segment from A(0,0) to B(2,1).Use the Bresenham's Line drawing algorithm to rasterize this line.

Step 1: Calculate

$$\Delta x = x_1 - x_0 = 2 - 0 = 2$$

$$\Delta y = y_1 - y_0 = 1 - 0 = 1$$

Step 2: Initialize decision parameter

$$p_k = 2\Delta y - \Delta x = 2(1) - 2 = 0$$

Step 3: Plot starting point

$$(x_0, y_0) = (0, 0)$$

Pixel plotted: **(0,0)**

Step 4: For each x until x_1

- Iteration 1:

  Current point = (0,0), $p_k = 0$.

  Since $p_k \geq 0$ → use **else case**:

$$(x_{k+1}, y_{k+1}) = (x_k + 1, y_k + 1) = (1, 1)$$

  Update decision parameter:

$$p_{k+1} = p_k + 2\Delta y - 2\Delta x = 0 + 2 - 4 = -2$$

https://t.me/SPPU_TE_BE_COMP

- **Iteration 2:**

  Current point = (1,1), $p_k = -2$.

  Since $p_k < 0$ → use **if case**:

$$(x_k + 1, y_k) = (x_k + 1, y_k) = (2, 1)$$

  Update decision parameter:

$$p_{k+1} = p_k + 2\Delta y = -2 + 2 = 0$$

Step 5: Generate remaining points

Step	(pk)	(x)	(y)	Plotted Point
0	0	0	0	(0, 0)
1	-2	1	1	(1, 1)
2	0	2	1	(2, 1)

Final Plotted Pixels

$$(0, 0) \rightarrow (1, 1) \rightarrow (2, 1)$$

https://t.me/SPPU_TE_BE_COMP

Comparison between DDA Line Drawing Algorithm and Bresenham's Line Drawing Algorithm

Sr. No.	DDA Line Drawing Algorithm	Bresenham's Line Drawing Algorithm
1	Uses floating–point arithmetic to calculate intermediate points along the line.	Uses integer arithmetic to determine the closest pixel to the line.
2	Involves addition, subtraction, and multiplication of floating–point numbers.	Primarily involves integer addition and subtraction.
3	Can introduce rounding errors because of the floating–point operations.	More accurate in plotting pixels close to the actual line path, with no rounding issues.
4	Slower due to the use of floating–point calculations, which require more processing power.	Faster, as it uses only integer arithmetic, making it more efficient on systems without floating–point hardware.
5	Simpler to understand and implement.	Slightly more complex due to the decision parameter and multiple cases for different slopes.
6	May produce lines with small deviations due to rounding, resulting in uneven pixel distribution.	Produces smooth lines with evenly distributed pixels, closely following the actual line.
7	Requires more memory due to floating–point operations.	Requires less memory due to the use of integer arithmetic.

Circle drawing algorithms:

1. Digital Differential Analyzer (DDA) Circle drawing algorithm.
2. Bresenham's Circle drawing algorithm.
3. Midpoint Circle Drawing algorithm

https://t.me/SPPU_TE_BE_COMP

1.Digital Differential Analyzer (DDA) Circle drawing algorithm.

- The Digital Differential Analyzer (DDA) algorithm for drawing a circle is a method used to plot the points of a circle in a rasterized form.
- The DDA algorithm is more commonly associated with line drawing, but it can be adapted for circles.
- However, the Bresenham's algorithm is typically preferred for circle drawing due to its efficiency.

Consider a circle centered at the origin (0,0):

$x^2 + y^2 = r^2$

DDA Algorithm Steps:

1. Initialize Parameters:

- Start from a point on the circle, typically at the topmost point (0, r).
- Set initial coordinates **x = 0** and **y = r**.

---

2. Calculate the Incremental Steps:

- Calculate the angle increment, **θ**, which is small enough to ensure that the points are close to each other:
  Δθ = 1 / r
- Calculate the incremental steps for **x** and **y**:
  Δx = cos(θ) × Δθ
  Δy = sin(θ) × Δθ

---

3. Iterate and Plot Points:

- For each step, update the coordinates using:
  x = x + Δx
  y = y − Δy
- Plot the pixel at **(x, y)**.
- Repeat until **x exceeds y**.

https://t.me/SPPU_TE_BE_COMP

4. Symmetry Consideration:

- Because of the circle's symmetry, you can mirror the points in **all eight octants** of the circle.
- This means that for each calculated point **(x, y)**, you can also plot the points:
  - **(y, x)**, **(−x, y)**, **(−y, x)**, and so on.

Bresenham's Circle Drawing Algorithm

Bresenham's Circle Drawing Algorithm is a rasterization algorithm used to draw a circle in computer graphics.

The algorithm is efficient because it uses only integer addition, subtraction, and bit shifting, making it much faster than other methods that involve floating–point arithmetic.

Key Concepts:

- **Symmetry**: A circle is symmetric about its center, which means that when you calculate one point on the circumference, you can easily find the corresponding points in the other seven octants of the circle.
- **Decision Parameter**: The algorithm uses a decision parameter to decide whether to move horizontally or diagonally at each step to approximate the circle.

Steps of the Algorithm:

---

1. Initial Setup:

- Consider a circle centered at (0,0) with radius **r**.
- Start from the topmost point **(0, r)**.
- The initial decision parameter $\mathbf{p_0}$ is calculated as:
  $\mathbf{p_0 = 3 - 2r}$
- This decision parameter helps in determining whether the next point will be directly to the right or diagonally down to the right.

https://t.me/SPPU_TE_BE_COMP

2. Iterative Calculation:

- For each point **(x, y)** on the circle's perimeter, the next point is determined based on the value of the decision parameter $\mathbf{p_k}$.
- **If $\mathbf{p_k < 0}$**, the next point is **(x + 1, y)** and the decision parameter is updated as:
  $\mathbf{p_{k+1} = p_k + 4x_k + 6}$
- **If $\mathbf{p_k \geq 0}$**, the next point is **(x + 1, y − 1)** and the decision parameter is updated as:
  $\mathbf{p_{k+1} = p_k + 4(x_k - y_k) + 10}$

3. Symmetry Exploitation:

- For every calculated point **(x, y)**, the corresponding points in all **eight octants** of the circle are plotted.
- The symmetry points are:
  (x, y), (y, x), (−x, y), (−y, x), (x, −y), (y, −x), (−x, −y), (−y, −x)

4. Stopping Condition:

- The algorithm continues iterating **until x becomes greater than or equal to y**.

https://t.me/SPPU_TE_BE_COMP

Example: Bresenham's circle algorithm with center (0,0) and radius 3

Start point:

$x = 0,\ y = 3$

Initial decision parameter:

$$P_0 = 3 - 2r = 3 - 6 = -3$$

- **Step 1:**

$P_0 = 3 - 2(3) = 3 - 6 = -3$

Since $P_0 < 0 \rightarrow$ use formula:

P_new = P + 4x + 6

Calculation for P_1:

$P_1 = -3 + 4(0) + 6$

$P_1 = -3 + 0 + 6$

$P_1 = 3$

Next values:

$x = x + 1 = 0 + 1 = 1$

$y = y = 3$

Next point: (1, 3)

https://t.me/SPPU_TE_BE_COMP

Step 2:

Current values: $x = 1, y = 3, P = P_1 = 3$

Since $P \geq 0 \rightarrow$ use formula:

P_new = P + 4(x − y) + 10

Calculation for P_2:

$P_2 = P_1 + 4(x - y) + 10$

$P_2 = 3 + 4(1 - 3) + 10$

$P_2 = 3 + 4(-2) + 10$

$P_2 = 3 - 8 + 10$

$P_2 = 5$

Next values:

$x = x + 1 = 1 + 1 = 2$

$y = y - 1 = 3 - 1 = 2$

Next point: (2, 2)

Step 3:

x = 2, y = 2

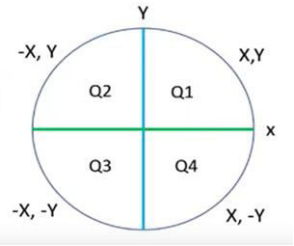
Now $x = y$, so stop —

(remaining points can be found using circle symmetry)

Points only for the **first octant** :

Step	xi	yi	Pi	xi+1	yi+1	Pi+1
1	0	3	-3	1	3	3
2	1	3	3	2	2	5
3	2	2	5	stop	stop	-

https://t.me/SPPU_TE_BE_COMP



The remaining points of the circle found using **symmetry from the first octant**

Q1 (X,Y)	Q2 (-X,Y)	Q3 (-X,-Y)	Q4 (X,-Y)
0,3	0,3	0,-3	0,-3
1,3	-1,3	-1,-3	1,-3
2,2	-2,2	-2,-2	2,-2
3,1	-3,1	-3,-1	3,-1
3,0	-3,0	-3,0	3,0

Midpoint Circle Drawing Algorithm

The Midpoint Circle Drawing Algorithm is another efficient way to draw a circle on a raster display. It is conceptually similar to Bresenham's Circle Drawing Algorithm but is derived directly from the midpoint method, which uses the midpoint between two candidate pixels to determine which one is closer to the actual circle.

Concept:
The Midpoint Circle Drawing Algorithm uses the symmetry of a circle to minimize the number of calculations. By calculating the points for one octant (1/8th of the circle), the algorithm can reflect those points into the other seven octants, significantly reducing the amount of work required.

https://t.me/SPPU_TE_BE_COMP

Steps of the Midpoint Circle Drawing Algorithm:

---

1. Circle Equation:

- The equation of a circle centered at the origin (0, 0) with radius **r** is:
  $\mathbf{x^2 + y^2 = r^2}$

---

2. Initial Setup:

- Start at the topmost point on the circle:
  $\mathbf{(x_0, y_0) = (0, r)}$
- Initialize the decision parameter:
  $\mathbf{p_0 = 1.25 - r}$
- This decision parameter helps determine whether the next pixel should be directly **East** or **South–East** of the current pixel.

---

3. Decision Making:

For each pixel, the algorithm needs to decide between two possible next points:

- **Directly to the right (East):**
  $\mathbf{(x + 1, y)}$
- **Diagonally down and to the right (South–East):**
  $\mathbf{(x + 1, y - 1)}$
- The decision is made based on the value of the decision parameter $\mathbf{p_k}$:
  - **If $\mathbf{p_k < 0}$**:
    The midpoint is **inside** the circle → Next point is **East**.
  - **If $\mathbf{p_k \geq 0}$**:
    The midpoint is **outside or on** the circle → Next point is **South–East**.

Polygons

Introduction to polygon

Polygons are fundamental shapes in geometry and computer graphics, defined as a **flat**, **two-dimensional shape** with **straight sides that are fully connected**, forming a **closed loop**.

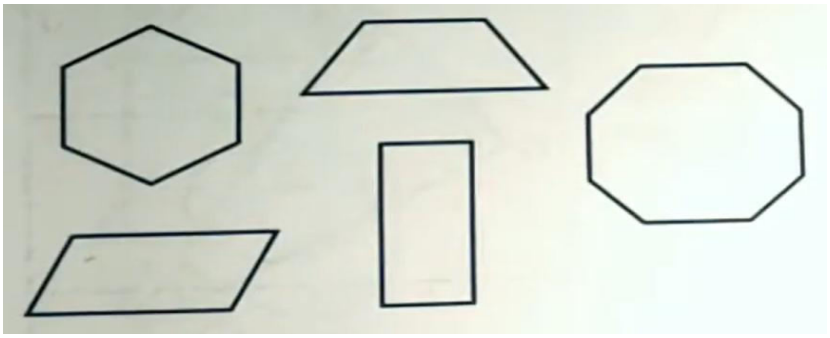
https://t.me/SPPU_TE_BE_COMP

Polygons can have **any number of sides**, but they must **have at least three**.

Types of Polygons

Convex Polygons

1. A **convex polygon** is a polygon in which all **interior angles are less than 180 degrees**.
2. This means that if you draw a line between any two points inside the polygon, the line will **never exit the polygon**.
3. Essentially, a convex polygon has **no indentations or "inward" curves**.



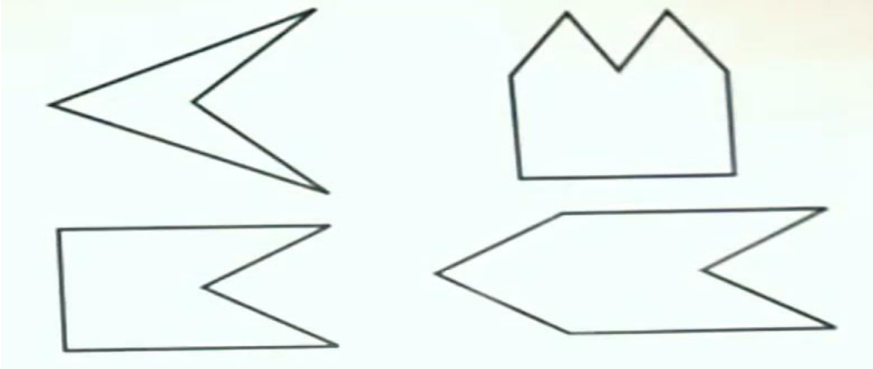
Characteristics of Convex Polygons:

- All interior angles are **less than 180 degrees.**
- Any **line segment drawn between two points** inside the **polygon lies entirely within the polygon.**
- **No internal angle "points inward**."
- Examples include regular polygons like **equilateral triangles, squares, and regular hexagons**

https://t.me/SPPU_TE_BE_COMP

Concave Polygons:

- A concave polygon is a polygon that has **at least one interior angle greater than 180 degrees.**
- A concave polygon will have **at least one line segment** that, when drawn between two points inside the polygon, will **pass outside of the polygon**.



Characteristics of Concave Polygons:

- At **least one interior angle is greater than 180 degrees**.
- There exists at least one line segment that can be drawn between two points inside the polygon that **passes outside of it**.
- The polygon has a "dent" or inward-pointing vertex.
- Examples include a star shape or an arrowhead shape.

Inside Test

The Even-Odd Method and the Winding Number Method are two algorithms used to determine whether a given point lies inside or outside a polygon.

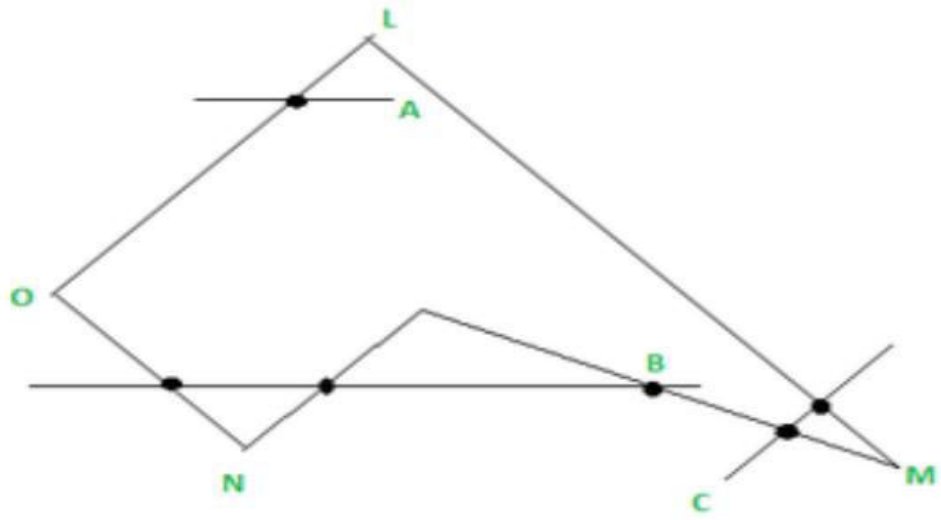
https://t.me/SPPU_TE_BE_COMP

1. Even-Odd Method:

The Even-Odd Method, also known as the Ray Casting Method, is based on counting how many times a ray extending from the point in question, intersects the edges of the polygon.

How It Works:

- Draw an imaginary ray from the point in question in any direction.
- Count the number of times this ray intersects the edges of the polygon.
    - If the number of intersections is odd, the point is inside the polygon.
    - If the number of intersections is even, the point is outside the polygon.



In the figure, the line segment from 'A' passes single edge and hence point A is inside the polygon. The point B is also inside the polygon as the line segment from B crosses three (odd) edges. But point C is outside the polygon it as the line segment from C crosses two (even) edges.

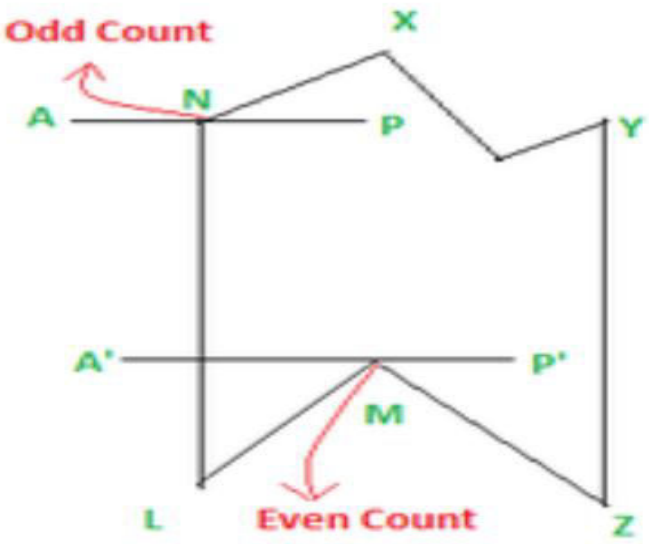
But this even-odd test fails when the intersection point is a vertex.

https://t.me/SPPU_TE_BE_COMP

To handle this case, we have to make few modifications.

We must look at the other end points of the two segments of a polygon which meet at this vertex. If these points lie on the same side of the constructed line AP, then the intersection point counts as an even number of intersections. But if they lie on the opposite side of constructed line AP, then the intersection points count as a single intersection.

As we can see the line segment A`P` intersects at M which is a vertex and L & Z are the other end points of the two segments meeting at M. L & Z lie on same side of the line segment A'P', so the count is taken as even.



2. Winding Number Method:

There is another alternative method for defining a polygons' interior point is called the winding number method. Conceptually, one can stretch a piece of elastic between the point (P) to be checked and the point on the polygon boundary.

https://t.me/SPPU_TE_BE_COMP

How It Works:

- Imagine traversing the boundary of the polygon in a consistent direction
- For each edge of the polygon, determine how it "winds" around the point:
    - If the edge starts below the line segment and finishes above, the edge has direction number '−1'
    - If the edge starts from above the line segment & finishes below the line, the edge has direction number '1'.

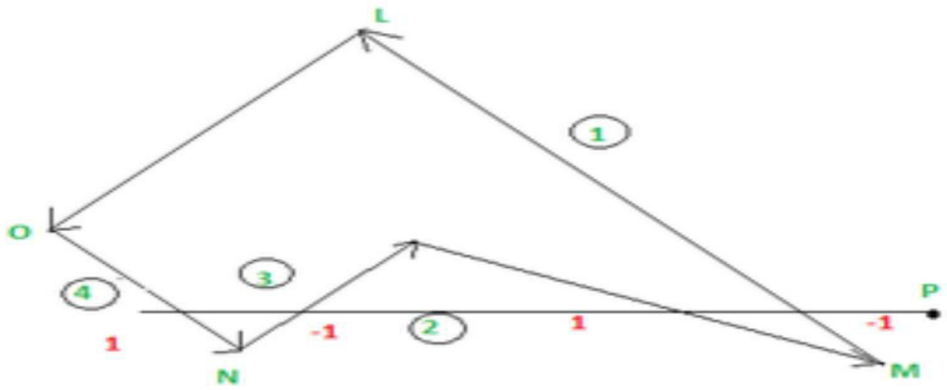
- The winding number is the sum of all the direction numbers.
- After considering all edges:
    - If the winding number is non-zero, the point is inside the polygon.
    - If the winding number is zero, the point is outside the polygon.

- Consider that an elastic is tied to point (P) to be checked, and the other end of the elastic is sliding along the boundary of the polygon until it has made one complete circuit.
- Then we check how many times the elastic has been wound around the point of intersection. If it is wound at-least once, point is inside. If there is no net winding then point is outside.
- In this method, instead of just counting the intersections, we give each boundary line crossed a direction number, and we sum these directions numbers.
- The direction number indicates the direction of the polygon edge that was drawn relative to the line segment we constructed for the test.

https://t.me/SPPU_TE_BE_COMP

Example: To test a point (xi, yi), let us consider a horizontal line segment y = yi which runs from outside the polygon to (xi, yi). We find all the sides which crossed this line segment.
Now there are 2 ways for side to cross, the side could be drawn starting below end, cross it and end above the line. In this case we can give direction numbers – 1, to the side or the edge could start above the line & finish below it in this case, given a direction 1.

The sum of the direction numbers for the sides that cross the constructed horizontal line segment yield the "Winding Number" for the point.
If the winding number is non-zero, the point is inside the polygon, else, the point is outside the
polygon.



In the above figure, the line segment crosses 4 edges having different direction numbers: 1, -1, 1 & −1 respectively, then:

Winding Number = 1 + (-1) + 1 + (-1) = 0

So, the point P is outside the Polygon. The edge has direction number −1 because it starts below the line segment & finishes above. Similarly, edge has direction number +1 because it starts from above the line segment & finishes below the line segment (See the directions in the figure).

https://t.me/SPPU_TE_BE_COMP

Polygon Filling

Seed Fill Method

Concept: Region filling starts from a seed point inside the polygon. From this seed, the algorithm spreads to neighbouring pixels and fills them until the region is complete.

It has two types:

1. Boundary Fill
2. Flood Fill

Boundary Fill

Boundary Fill is a polygon filling algorithm used in computer graphics to fill a region defined by a specific boundary. The algorithm starts from a seed point inside the region and spreads outwards, filling pixels until it reaches the boundary. The boundary is typically defined by a specific color, and the algorithm stops filling when it encounters this boundary color.

How Boundary Fill Works:

1. Seed Point:
The algorithm begins at a seed point, which is a pixel inside the area to be filled.

2. Filling Process:

- The algorithm checks the color of the current pixel. If the pixel is not the boundary color and not the fill color, it changes the pixel to the fill color.
- The algorithm then recursively or iteratively moves to the neighboring pixels and repeats the process.

https://t.me/SPPU_TE_BE_COMP

3. Boundary Detection:

- The filling process continues until the algorithm encounters the boundary color, at which point it stops filling in that direction.

Types of Boundary Fill:

1. 4-Connected Boundary Fill:

- This method checks and fills the four neighboring pixels (left, right, up, down) of the current pixel.
- It is simple but may leave gaps if the boundary is diagonal.

Steps:

1. Start from the seed point.
2. If the current pixel's color is neither the boundary color nor the fill color, change it to the fill color.
3. Move to the left, right, up, and down neighbors and repeat the process.
4. Stop when all connected pixels within the boundary are filled.

2. 8-Connected Boundary Fill:

- This method checks and fills the eight neighboring pixels (left, right, up, down, and the four diagonals).
- This approach ensures that all areas within the boundary, including diagonallyconnected regions, are filled.

Steps:

1. Start from the seed point.
2. If the current pixel's color is neither the boundary color nor the fill color, change it to the fill color.
3. Move to the eight neighboring pixels and repeat the process.
4. Stop when all connected pixels within the boundary are filled.

https://t.me/SPPU_TE_BE_COMP

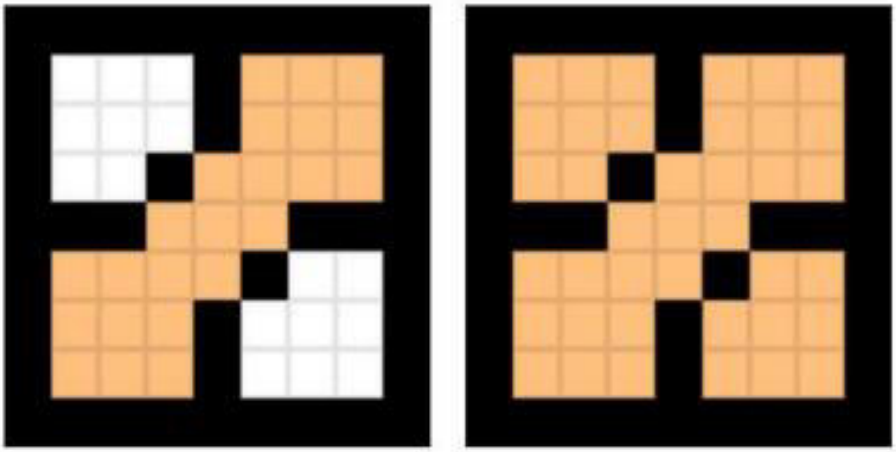
8-Connected Boundary Fill Algorithm

```
Boundary_fill(int x, int y, int fillcolor, int bcolor)
{
    if (getpixel(x, y) != bcolor && getpixel(x, y) != fillcolor)
    {
        putpixel(x, y, fillcolor);

        // 8 directions: right, down, left, up, and 4 diagonals
        Boundary_fill(x + 1, y, fillcolor, bcolor);     // Right
        Boundary_fill(x - 1, y, fillcolor, bcolor);     // Left
        Boundary_fill(x, y + 1, fillcolor, bcolor);     // Down
        Boundary_fill(x, y - 1, fillcolor, bcolor);     // Up

        Boundary_fill(x + 1, y + 1, fillcolor, bcolor); // Down-Right
        Boundary_fill(x - 1, y + 1, fillcolor, bcolor); // Down-Left
        Boundary_fill(x + 1, y - 1, fillcolor, bcolor); // Up-Right
        Boundary_fill(x - 1, y - 1, fillcolor, bcolor); // Up-Left
    }
}
```

Example: Imagine you have a region bounded by a black line, and the area inside this boundary is white. You want to fill this white area with red.



The 4-connected method fails to fill this figure completely. This figure will be efficiently filled using the 8-connected technique.

https://t.me/SPPU_TE_BE_COMP

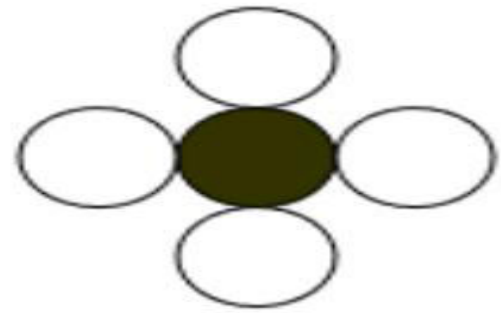
Flood Fill Algorithm:

Flood Fill is a classic algorithm used in computer graphics for filling an area bounded by a polygon or another region with a specific color. It is commonly used in paint programs and graphic editors for operations like filling enclosed areas with a solid color or pattern.

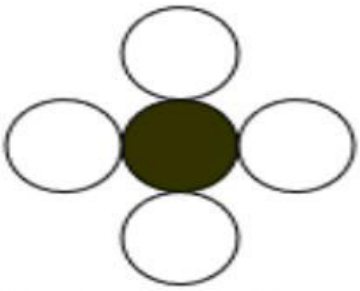
How Flood Fill Works:

Flood Fill works by starting from a seed point (a pixel inside the area to be filled) and then spreading out to fill neighboring pixels that meet a specific criterion (such as having the same color as the seed point).

Types of Flood Fill Algorithms:



4- Connected



4- Connected

1.4-Connected Flood Fill:

- This method checks the four neighboring pixels (up, down, left, right) of the current pixel.
- The algorithm recursively or iteratively fills the neighboring pixels if they match the target color (the color to be replaced).
- This approach is simple but might leave gaps if the region to be filled has diagonal connections.

https://t.me/SPPU_TE_BE_COMP

Steps:

1. Start from the seed point.
2. If the pixel's color matches the target color, change it to the fill color.
3. Move to the neighboring pixels (left, right, up, down) and repeat the process.
4. Stop when all connected pixels with the target color are filled.

2. 8-Connected Flood Fill:

- This method is similar to the 4-connected method but additionally checks the diagonal neighbors.
- The algorithm ensures that all pixels connected diagonally, as well as horizontally and vertically, are filled.

Steps:

1. Start from the seed point.
2. If the pixel's color matches the target color, change it to the fill color.
3. Move to all eight neighboring pixels (left, right, up, down, and the four diagonals) and repeat the process.
4. Stop when all connected pixels with the target color are filled.

https://t.me/SPPU_TE_BE_COMP

Flood Fill Algorithm (8-connected)

```
void flood-fill(int x, int y, int oldcolor, int newcolor)
{
    // Check if the current pixel matches the color we want to replace
    if (getpixel(x, y) == oldcolor)
    {
        // Yes, this is the pixel we need to fill
        putpixel(x, y, newcolor);  // Fill the current pixel with the new color

        // Now recursively check all 4-connected neighbors
        flood-fill(x + 1, y, oldcolor, newcolor); // Right neighbor
        flood-fill(x - 1, y, oldcolor, newcolor); // Left neighbor
        flood-fill(x, y - 1, oldcolor, newcolor); // Top neighbor
        flood-fill(x, y + 1, oldcolor, newcolor); // Bottom neighbor

        // Also check all 4 diagonal neighbors – for 8-connected fill
        flood-fill(x + 1, y + 1, oldcolor, newcolor); // Bottom-right neighbor
        flood-fill(x + 1, y - 1, oldcolor, newcolor); // Top-right neighbor
        flood-fill(x - 1, y + 1, oldcolor, newcolor); // Bottom-left neighbor
        flood-fill(x - 1, y - 1, oldcolor, newcolor); // Top-left neighbor
    }
}
```

Advantages of 8-Connected Flood Fill over the 4-Connected Flood Fill

1. **Comprehensive Diagonal Coverage:** The 8-Connected Method considers diagonal neighbors in addition to horizontal and vertical ones, ensuring that all connected pixels, including those diagonally adjacent, are filled.
2. **More Complete Fill for Irregular Shapes**: This method is better suited for filling irregular shapes, especially those with diagonal components, ensuring that no gaps are left unfilled.
3. **Better Handling of Thin Lines and Corners**: The 8-Connected Method can accurately fill thin lines and sharp corners, which

https://t.me/SPPU_TE_BE_COMP

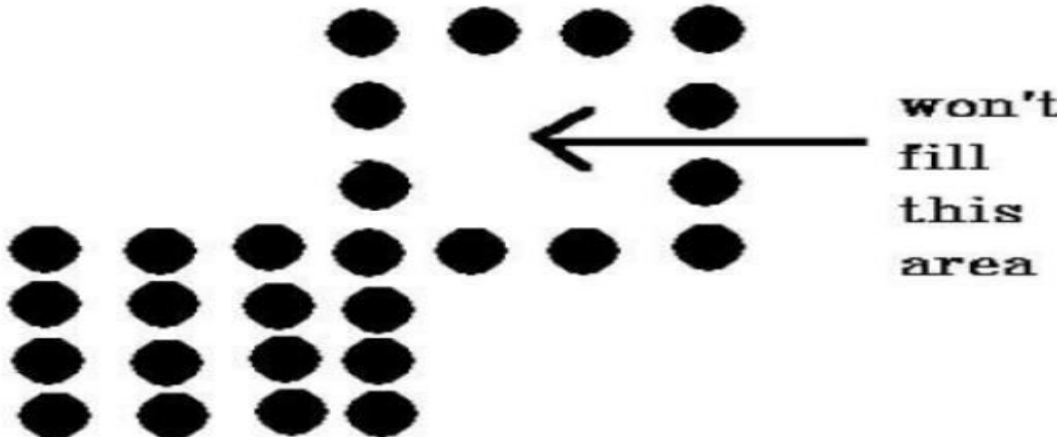
might be missed or only partially filled by the 4-Connected Method.

4. **Natural and Accurate Fill Appearance**: The fill produced by the 8-Connected Method tends to look more natural and complete, especially in complex regions, leading to a smoother and more accurate representation of the filled area.
5. **Avoidance of Unintended Gaps:** By considering diagonal connections, the 8-Connected Method prevents the creation of unintended gaps that might occur in regions where diagonal pixels are part of the shape or boundary.

Example:

Imagine filling a region using the 4-Connected Method, the algorithm would only fill the central pixels, leaving the diagonally connected parts unfilled as shown in the figure below.

The 8-Connected Method, however, would fill the entire shape, as it takes diagonal connections into account as well.



https://t.me/SPPU_TE_BE_COMP

Comparison between Flood-fill and Boundary-fill Algorithm

Sr No.	Flood-fill Algorithm	Boundary-fill Algorithm
1	It can process the image containing more than one boundary colors.	It can only process the image containing single boundary color.
2	Flood-fill algorithm is comparatively slower than the Boundary-fill algorithm.	Boundary-fill algorithm is faster than the Flood-fill algorithm.
3	In Flood-fill algorithm a random color can be used to paint the interior portion, then the old one is replaced with a new one.	In Boundary-fill algorithm, interior points are painted by continuously searching for the boundary color.
4	It requires huge amount of memory.	Memory consumption is relatively low in Boundary-fill algorithm.
5	Flood-fill algorithms are simple and efficient.	The complexity of Boundary-fill algorithm is high.

Scan-Line Fill

Scan-Line Fill is a widely used polygon filling algorithm in computer graphics, especially for rendering filled polygons on raster displays. It operates by processing the polygon one horizontal line (or scan line) at a time, determining which parts of the scan line lie inside the polygon, and then filling those parts with the desired color.

How Scan-Line Fill Works:

The algorithm works by considering the intersections of the polygon's edges with each scan line. Here's the step-by-step process:

https://t.me/SPPU_TE_BE_COMP

1. Initialization:

- Determine the bounding box of the polygon (i.e., the minimum and maximum x and y coordinates that enclose the polygon).
- Start with the lowest y-coordinate of the bounding box and process each scan line up to the highest y-coordinate.

2. Edge Intersections:

- For each scan line, determine the intersection points between the scan line and the edges of the polygon.
- This is typically done by solving the line equation for each edge of the polygon to find where the scan line intersects the edge.

3. Sort Intersections:

- The intersection points on the scan line are sorted by their x-coordinates.

4. Filling Between Intersections:

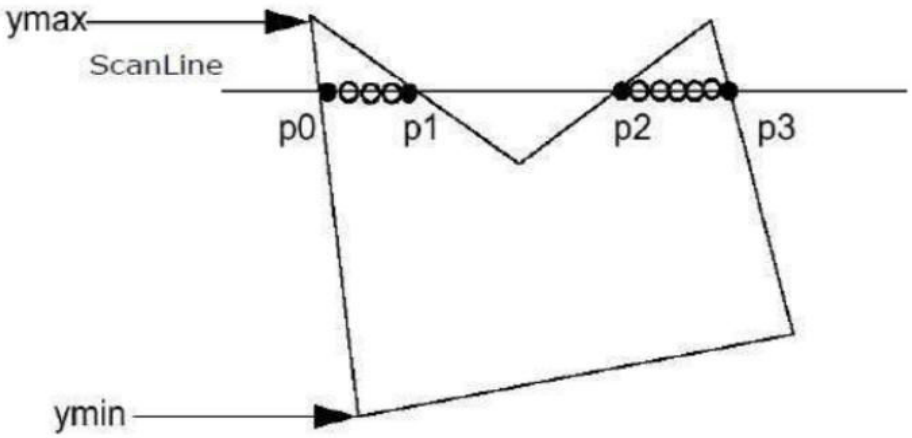
- After sorting, the algorithm fills the pixels between pairs of intersection points.
- If there are multiple pairs of intersections, alternate between filling and not filling the sections, because only the even-numbered intervals (inside the polygon) need to be filled.

5. Handling Special Cases:

- If a scan line passes exactly through a vertex, careful handling is required to avoid counting the vertex twice, which could lead to an incorrect fill.
- Shared vertices or edges that lie on the scan line should only be considered once.

https://t.me/SPPU_TE_BE_COMP

Example of Scan-Line Fill:



Steps for Algorithm

Step 1: Find out the ymin and ymax from the given polygon.

Step 2: Scan Line intersects with each edge of the polygon from ymin to ymax. Name each intersection point of the polygon. As per the figure shown above, they are named as p0, p1, p2,p3.

Step 3: Sort the intersection point in the increasing order of x coordinate i.e. (p0, p1), (p1, p2)and (p2, p3).

Step 4: Fill all those pairs of coordinates that are inside polygons and ignore the alternate pairs.

Windowing and clipping:

Viewport Transformations

Viewing transformations are a set of operations in computer graphics that map a scene from world coordinates to a device's screen coordinates. These transformations define how a 3D scene is projected onto a 2D viewing surface (like a computer screen), allowing us to view

https://t.me/SPPU_TE_BE_COMP

a 3D object from different perspectives. The process involves several key steps:

Steps in Viewing Transformation:

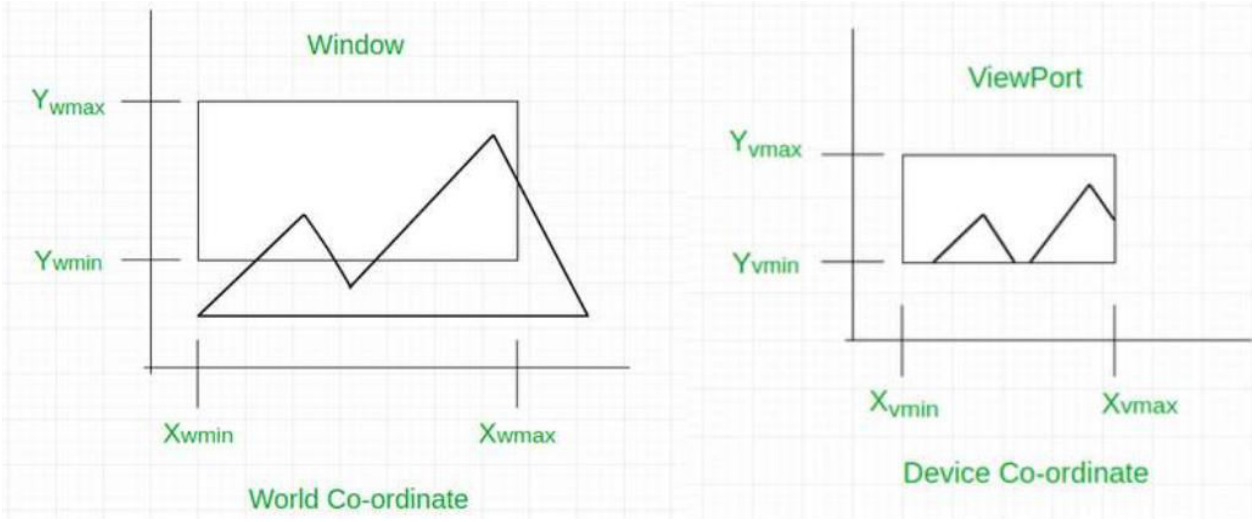
1. Modeling Transformation:
- Transforms the object coordinates to world coordinates. This step involves translating, scaling, and rotating the object within the world coordinate system.

3. Viewing Transformation:
- Transforms the world coordinates to viewing coordinates. This aligns the scene with the viewer's position and orientation.

4. Projection Transformation:
- Projects the viewing coordinates onto the projection plane, creating a 2D representation of the 3D scene.

5. Clipping and Normalization:
- Removes parts of the scene outside the view volume and normalizes the coordinates within a standard range.

5. Viewport Transformation:
- Maps the normalized coordinates to the device's viewport, where they are finally displayed.



https://t.me/SPPU_TE_BE_COMP

General Terms:

- **World Coordinate**: It is the Cartesian coordinate with respect to which we define the diagram,like xwmin, xwmax, ywmin, ywmax.
- **Device Coordinate**: It is the screen coordinate where the objects are to be displayed,like xvmin, xvmax, yvmin, yvmax.
- **Window**: It is the area on the world coordinate selected for display.
- **Viewport:** It is the area on the device coordinate where graphics is to be displayed

2D Clipping

2D Clipping is a process in computer graphics that involves restricting the rendering of objects or parts of objects (like lines, polygons, or text) to a specific rectangular region known as the clipping window or clipping rectangle**.**

Any part of an object outside this clipping window is "clipped" away, meaning it is not displayed.

The purpose of 2D clipping is to ensure that only the visible portions of objects are drawn on the screen, which optimizes rendering and maintains focus on the relevant part of the scene.

1 Cohen-Sutherland Line Clipping Algorithm

The Cohen-Sutherland Line Clipping Algorithm is a popular method used in computer graphics for clipping a line segment to a rectangular clipping window.

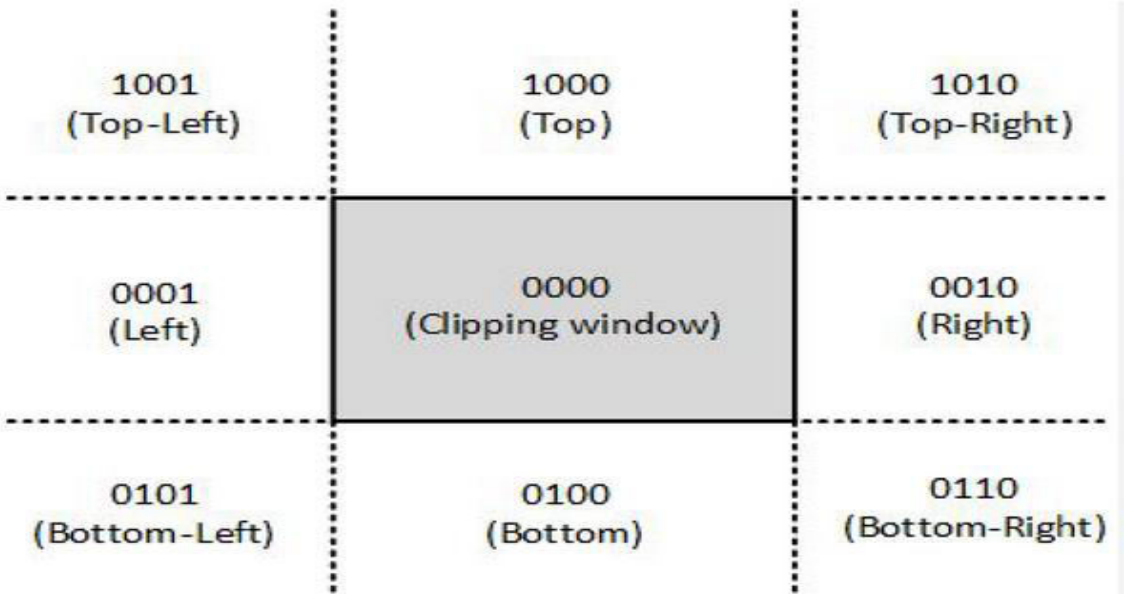
https://t.me/SPPU_TE_BE_COMP

Description:

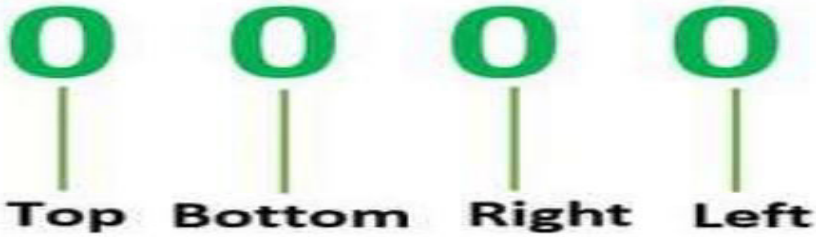
In this algorithm, we are given 9 regions on the screen. Out of which one region is of the window and the rest 8 regions are around it given by 4-digit binary. The division of the regions are based on (xmax, ymax) and (xmin, ymin).

The central part is the viewing region or window, all the lines which lie within this region are completely visible. A region code is always assigned to the endpoints of the given line.

To check whether the line is visible or not.



Formula to check binary digits: TBRL which can be defined as top, bottom, right, and left accordingly.



https://t.me/SPPU_TE_BE_COMP

Steps of the Cohen-Sutherland Line Clipping Algorithm:

1. Assign Outcodes:
Compute the outcodes for both endpoints of the line segment.

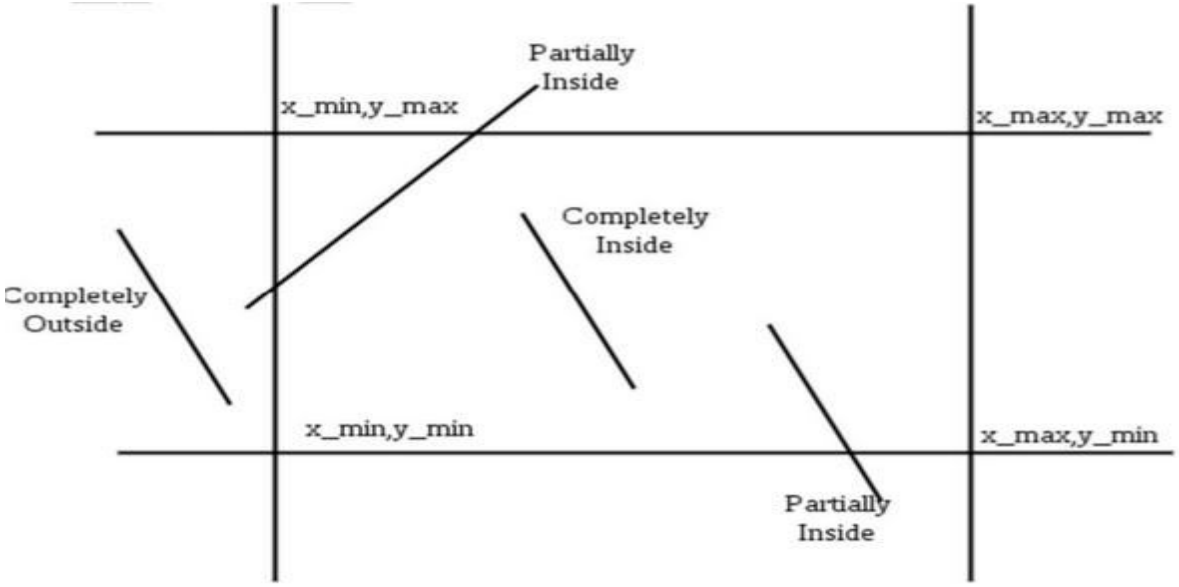
2. Line Completely Inside:
If both outcodes are 0000, the line segment is completely inside the clipping window,and the line can be accepted without further processing.

3. Line Completely Outside:
If the logical AND operation between the two outcodes is not 0000, the line segmentis entirely outside the clipping window, and the line can be rejected.

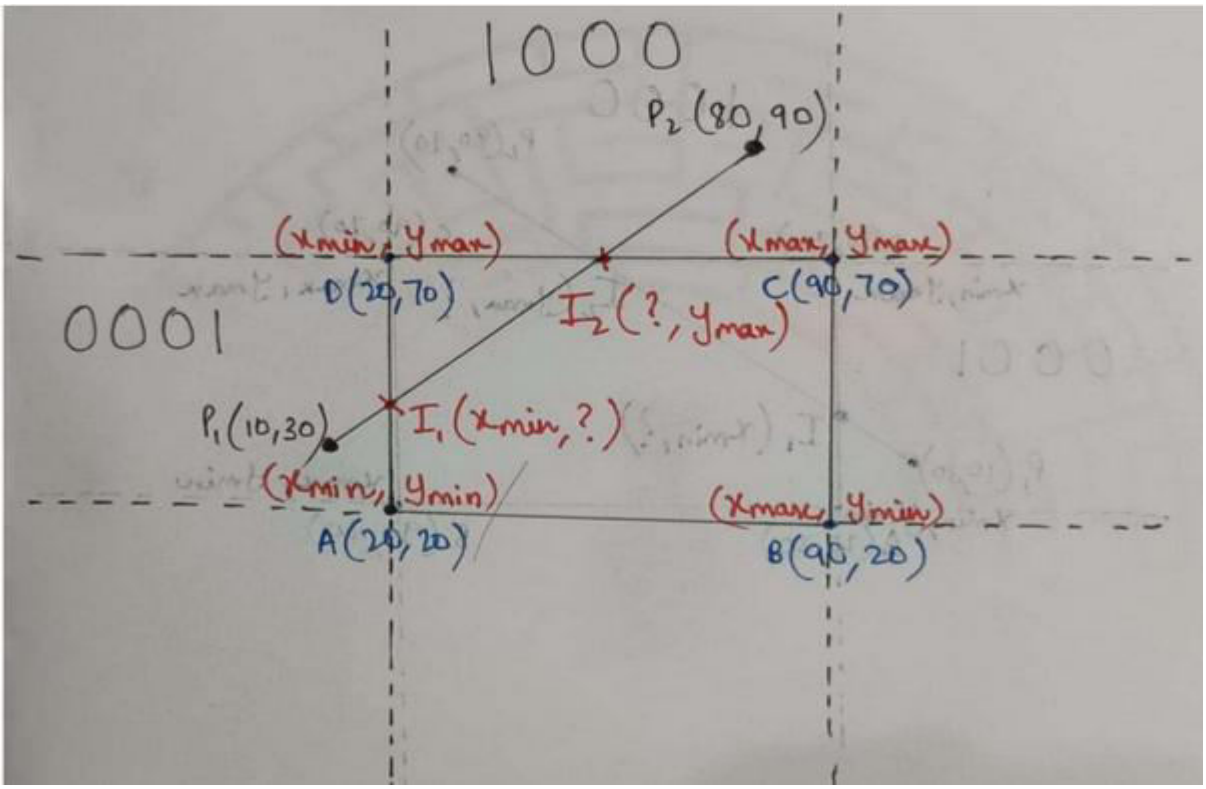
4. Line Partially Inside:

- If the **line cannot be trivially accepted or rejected**, the line **intersects the clipping window**, and **further processing is needed to find the intersection points.**
- Determine which **bit is set in the outcode** of the point **outside the clipping window and calculate the intersection of the line with the corresponding clipping boundary.**
- **Replace the outside endpoint with the intersection point** and **update its outcode.**
- Repeat the process until the line is either **trivially accepted or rejected**

https://t.me/SPPU_TE_BE_COMP



Example: Let ABCD be the rectangular window with A(20, 20), B(90, 20), C(90, 70), and D(20, 70). Find region codes for endpoints and use the Cohen-Sutherland algorithm to clip the lines: (i) P1 P2 with P1 (10, 30), P 2 (80, 90).



https://t.me/SPPU_TE_BE_COMP

P1 = 0001, P2 = 1000

AND of P1 and P2 = 0000

Hence, the line is partially visible.

Slope, m =

$$m = \frac{y_2 - y_1}{x_2 - x_1} = \frac{90 - 30}{80 - 10} = \frac{60}{70} = \frac{6}{7}$$

For 1st intersection point I_1:

$$y = m(x - x_1) + y_1$$

$$y = m(x_{min} - x_1) + y_1$$

$$y = 0.857(20–10) + 30 = 8.57 + 30$$

$$y = 38.57 \approx 39$$

$$I_1(x_{min}, y) = (20, 39)$$

For 2nd intersection point:

$$x = \frac{1}{m}(y - y_1) + x_1$$

$$x = \frac{1}{m}(y_{max} - y_1) + x_1$$

$$x = \frac{1}{0.857}(70 - 30) + 10 = \frac{40}{0.857} + 10$$

$$x = 56.67$$

$I_2\ (x, y_{max}) = (57, 70)$

https://t.me/SPPU_TE_BE_COMP

2. Sutherland Hodgeman Polygon Clipping Algorithm

The Sutherland-Hodgman Polygon Clipping Algorithm is a widely used method in computer graphics for clipping a polygon against a rectangular clipping window. This algorithm processes each edge of the clipping window and modifies the vertices of the polygon to ensure that only the portion inside the clipping window is retained.

Steps of the Sutherland-Hodgman Polygon Clipping Algorithm:
The algorithm processes the polygon vertices by clipping the polygon against each of the four clipping window edges (left, right, bottom, and top) one by one. After processing against one edge, the resulting polygon is used as input for the next edge.

1. Read coordinates of all vertices of the polygon.
2. Read coordinates of the clipping window
3. Consider the left edge of the window
4. Compare the vertices of each edge of the polygon, individually with the clipping plane
5. Save the resulting intersections and vertices in the new list of vertices according to four possible relationships between the edge and the clipping boundary discussed earlier.
6. Repeat the steps 4 and 5 for remaining edges of the clipping window. Each time the resultant list of vertices is successively passed to process the next edge of the clipping window.
7. Stop.

There are four possible cases for any given edge of given polygon against current clipping edge.

https://t.me/SPPU_TE_BE_COMP

1. Both vertices are inside: Only the second vertex is added to the output list.

2. First vertex is outside while second one is inside: Both the point of intersection of the edge with the clip boundary and the second vertex are added to the output list.

3. First vertex is inside while second one is outside: Only the point of intersection of the edge with the clip boundary is added to the output list.

4. Both vertices are outside: No vertices are added to the output list.

Example: Let us consider a polygon and the clipping window as shown in figure:

This figure illustrates the **polygon clipping process**, where a polygon is cut to fit inside a specified rectangular region known as the **clipping window**.

The method shown is a step-by-step **Sutherland–Hodgman polygon clipping algorithm**, which clips the polygon against each boundary (left, right, top, and bottom) of the clipping window one at a time

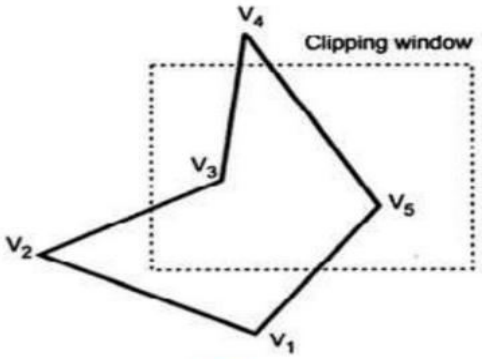


Fig. Before Clipping

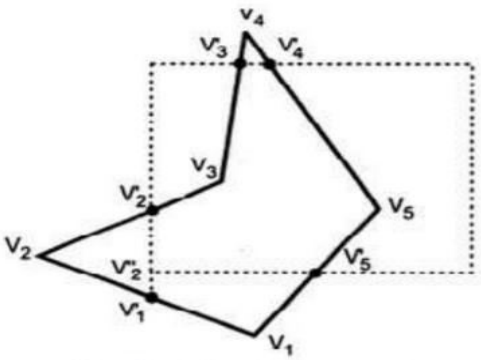


Fig. After Boundary Clipping

1. Before Clipping (Left Figure)

The polygon has vertices **V_1, V_2, V_3, V_4, V_5.**

The dotted rectangle is the **clipping window.**

Some vertices — **V_1, V_2, and V_4** — lie **outside** the window, while **V_3 and V_5** lie **inside**

https://t.me/SPPU_TE_BE_COMP

So, part of the polygon goes beyond the window boundaries.

2. After Boundary Clipping (Right Figure)

This figure shows the polygon **after clipping** it against all four boundaries (Left, Right, Top, and Bottom) of the window one by one.

a) Left Clipping

- The left edge of the window cuts the polygon.
- New intersection points V_1' and V_2' are created where the edges cross the left boundary.
- After this step, visible vertices are $V_1', V_2', V_3, V_4, V_5, V_1$.

b) Right Clipping

- The right edge of the window clips any part going beyond it.
- In this figure, no extra points are created because the polygon already fits within the right side.
- Vertices remain $V_1', V_2', V_3, V_4, V_5, V_1$.

c) Top Clipping

- The top boundary removes the part of the polygon above the window.
- New intersection points V_3' and V_4' are formed where the polygon crosses the top boundary.
- Now the visible vertices are $V_1', V_2', V_3, V_3', V_4', V_4, V_5, V_1$.

d) Bottom Clipping

- The bottom edge removes the portion of the polygon below the window.
- A new intersection point V_5' appears where the polygon crosses the bottom boundary.
- The final visible polygon lies completely inside the window with vertices $V_1', V_2', V_3, V_3', V_4', V_4, V_5, V_5', V_1$.

https://t.me/SPPU_TE_BE_COMP